

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

**Отчёт
Лабораторная работа №3**

Выполнил:
Степанов Виктор
J3213

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Мигрировать ранее разработанное приложение (ЛР1 и ЛР2) на фреймворк Vue.js. Требования: подключённый роутер, работа с внешним API через axios, разумное деление на компоненты, использование composable для повторяющегося функционала.

Теоретическая часть

Что такое Vue.js

Vue.js — прогрессивный JavaScript-фреймворк для построения пользовательских интерфейсов. В отличие от ЛР1 (ванильный JS), Vue автоматически обновляет DOM при изменении данных.

Ключевое отличие от ЛР1: в ЛР1 нужно вручную вызывать `document.getElementById()` и менять HTML. В Vue достаточно изменить переменную — шаблон обновится сам.

Компонент

Компонент — переиспользуемый блок интерфейса в файле `.vue`. Каждый файл содержит три секции:

```
<template>
  <!-- HTML                                -->
  <div>{{ message }}</div>
</template>

<script setup>
// JavaScript
import { ref } from 'vue'
const message = ref('')
</script>

<style scoped>
/* CSS                                */
div { color: blue; }
</style>
```

Зачем компоненты? `ModelCard.vue` используется на главной, в каталоге и в кабинете. Изменение в одном файле применяется везде.

Реактивность: ref и computed

`ref()` — создаёт реактивную переменную. При изменении Vue перерисовывает шаблон:

```
const models = ref([]) //
models.value = await getModels() // Vue
```

`computed()` — вычисляемое свойство, пересчитывается автоматически при изменении зависимостей:

```
const topModels = computed(() =>
  models.value
```

```

    .filter(m => m.type === 'model')
    .sort((a, b) => b.stars - a.stars)
    .slice(0, 3)
  )

```

Props — передача данных в компонент

Props — данные которые передаются в компонент снаружи. Компонент объявляет что принимает, родитель передаёт:

```

// ModelCard.vue
defineProps({
  model: { type: Object, required: true },
  col: { type: String, default: 'col-md-4' }
})

```

```

<!-- -->
<ModelCard :model="m" col="col-md-6" />

```

Компонент **не может изменять props** — только читать.

Хуки жизненного цикла

`onMounted()` — выполняется когда компонент появился на странице. Запросы к API делают именно здесь:

```

onMounted(async () => {
  models.value = await getModels()
  // DOM
})

```

Vue Router — маршрутизация

Vue Router управляет навигацией в SPA без перезагрузки страницы. URL меняется, но браузер не делает новый запрос к серверу — Vue просто показывает нужный компонент:

```

const routes = [
  { path: '/', component: HomeView },
  { path: '/search', component: SearchView },
  { path: '/model/:id', component: ModelView },
  { path: '/login', component: LoginView },
  { path: '/dashboard', component: DashboardView,
    meta: { requiresAuth: true } }
]

```

`meta: { requiresAuth: true }` — маршрут защищён. Navigation guard проверяет это перед каждым переходом:

```

router.beforeEach((to, from, next) => {
  const token = localStorage.getItem('token')
  if (to.meta.requiresAuth && !token) {
    next('/login') //
  }
})

```

```

    } else {
      next()
    }
  })
}

```

Проверить navigation guard: выйти из аккаунта и открыть localhost:5173/dashboard — автоматически перенаправит на /login.

Composable — переиспользование логики

Composable — функция которая выносит реактивную логику в отдельный файл. Позволяет использовать одно состояние в разных компонентах.

useAuth.js — логика авторизации:

```

const user = ref(null) //
const loggedIn = ref(false)

export function useAuth() {
  async function login(email, password) {
    const { data } = await API.post('/login',
      { email, password })
    localStorage.setItem('token', data.token)
    user.value = data.user
    loggedIn.value = true
  }
  return { user, loggedIn, login, logout,
    register, checkSession }
}

```

user и loggedIn — одни переменные для всего приложения. App.vue показывает имя пользователя в navbar, DashboardView отображает данные профиля — оба используют useAuth() и видят одно и то же состояние.

useApi.js — запросы к API моделей с автоматическим токеном:

```

API.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization =
    'Bearer ${token}'
  return config
})

export function useApi() {
  async function getModels() {
    const { data } = await API.get('/models')
    return data
  }
  async function getModel(id) {
    const { data } = await API.get(`/models/${id}`)
    return data
  }
  return { getModels, getModel }
}

```

Interceptor — перехватчик запросов в axios. Автоматически добавляет токен к каждому запросу. Не нужно писать заголовок вручную каждый раз.

axios vs fetch

В ЛР2 использовался встроенный `fetch`. В ЛР3 — `axios`:

- Автоматически парсит JSON (не нужно `.json()`)
- Поддерживает interceptors (перехват запросов/ответов)
- Удобная обработка ошибок через `e.response?.data?.error`
- Создание экземпляра с базовым URL: `axios.create({ baseURL })`

SPA vs обычный сайт

Обычный сайт: при переходе браузер запрашивает новую HTML-страницу с сервера. Каждый переход — полная перезагрузка.

SPA (Single Page Application): одна HTML-страница загружается один раз. Роутер меняет компоненты без обращения к серверу. Быстро, плавно, переходы мгновенные.

В ЛР1 была имитация SPA — все страницы в одном HTML, JS скрывал/показывал `div`. В ЛР3 — настоящий SPA на Vue Router с историей браузера (`createWebHistory`).

Ход работы

Структура проекта

Проект создан через `npm create vue@latest lab3`. Структура:

- `src/main.js` — точка входа: создаёт Vue-приложение, подключает роутер
- `src/App.vue` — корневой компонент: `navbar` с авторизацией, темой, `<RouterView/>`
- `src/router/index.js` — маршруты и `navigation guard`
- `src/composables/useApi.js` — `axios`-запросы к API моделей
- `src/composables/useAuth.js` — состояние авторизации и методы входа/выхода
- `src/components/ModelCard.vue` — переиспользуемая карточка модели
- `src/views/HomeView.vue` — главная страница
- `src/views/SearchView.vue` — каталог с фильтрацией
- `src/views/ModelView.vue` — страница модели
- `src/views/LoginView.vue` — форма входа
- `src/views/RegisterView.vue` — форма регистрации
- `src/views/DashboardView.vue` — личный кабинет

Запуск проекта

Для работы нужно два запущенных сервера:

Сервер 1 — API (json-server):

```
cd labs/lab2
npm start
# JSON Server      http://localhost:3001
```

Сервер 2 — Vue (Vite):

```
cd labs/lab3
npm run dev
# VITE              http://localhost:5173
```

Открыть в браузере: <http://localhost:5173>

Отличия от ЛР1 и ЛР2

- **ЛР1** — один HTML-файл, захардкоженные данные, фиктивная авторизация
- **ЛР2** — тот же HTML, но данные из json-server, реальная авторизация через токен
- **ЛР3** — полностью переписано на Vue.js: компоненты, роутер, composables, axios

Логика осталась той же, изменился способ её реализации.

Результаты

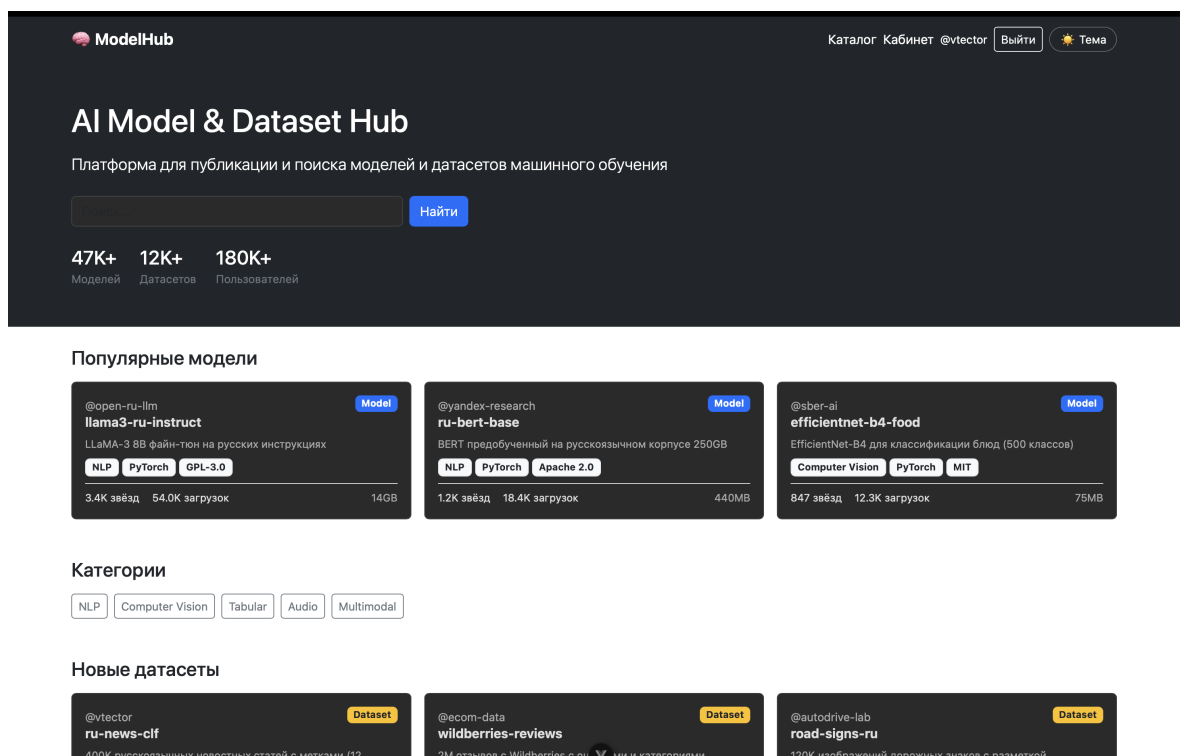


Рис. 1: Главная страница. Компонент HomeView.vue. Карточки — компонент ModelCard.vue, загружаются через useApi composable.

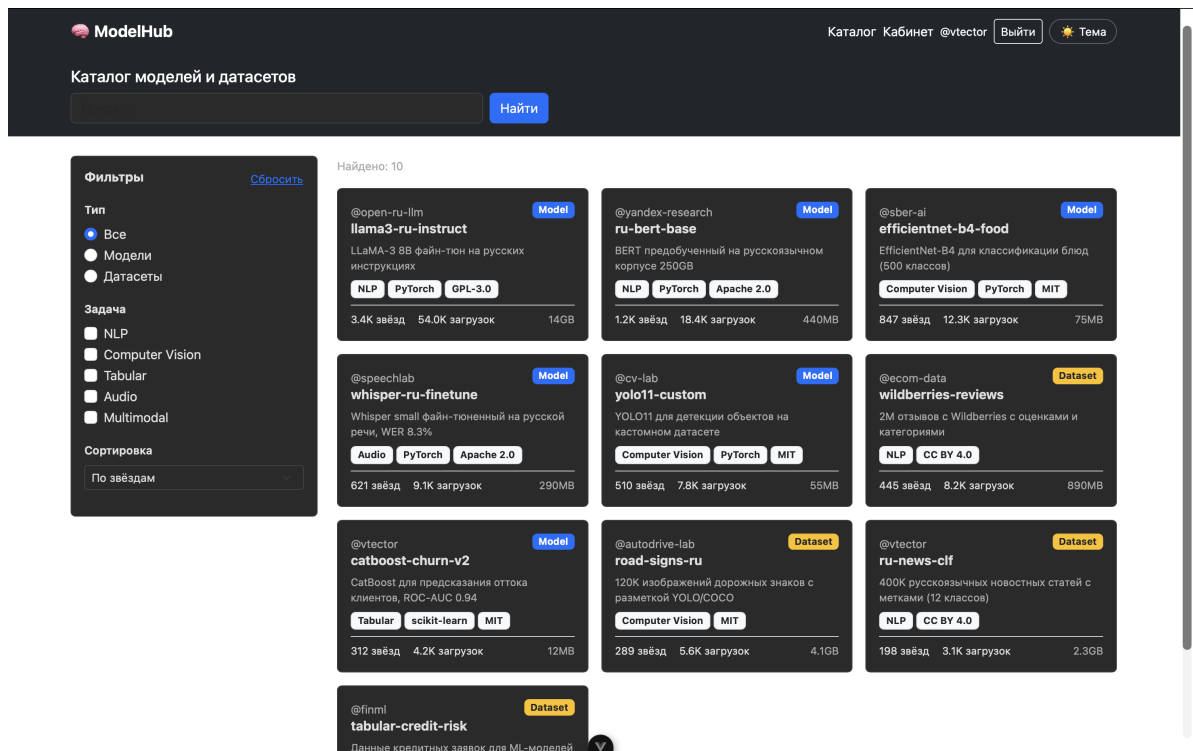


Рис. 2: Каталог. SearchView.vue. Фильтрация через computed() на фронте, без повторных запросов к серверу.

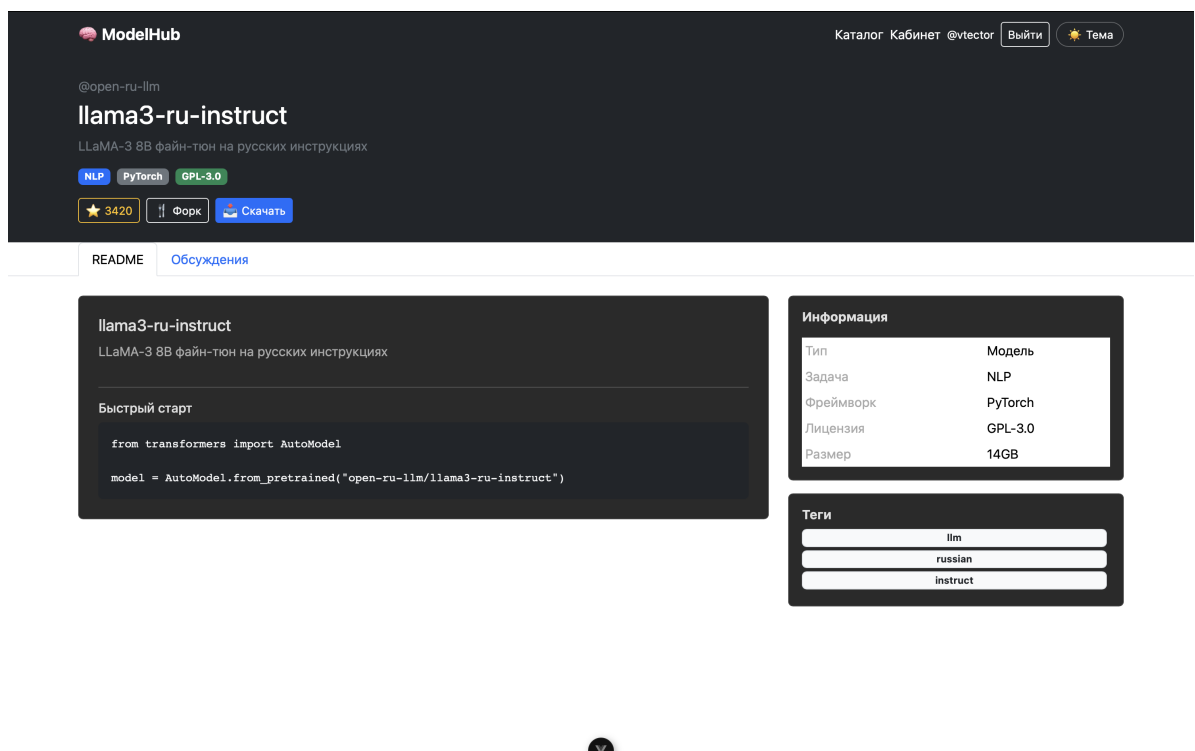


Рис. 3: Страница модели. ModelView.vue. Данные загружаются через useApi.getModel(id), где id берётся из URL через useRoute().

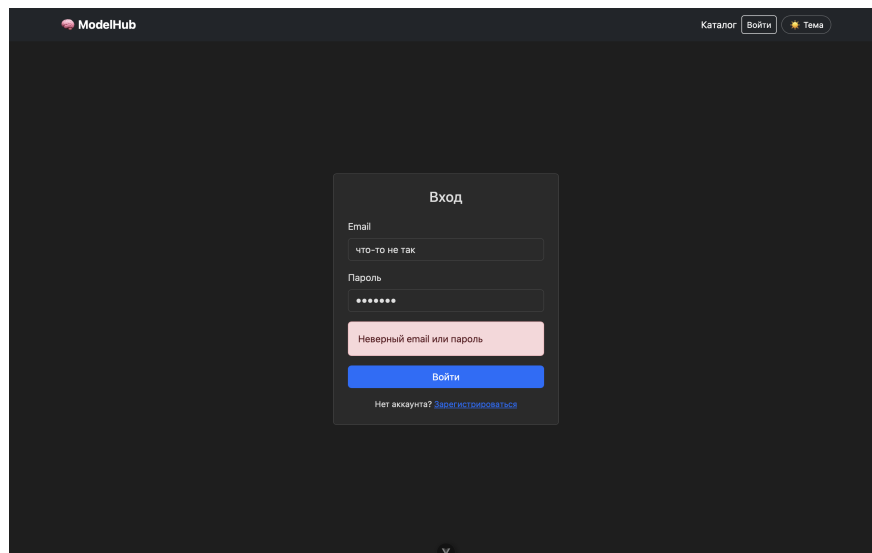


Рис. 4: Форма входа. LoginView.vue. При ошибке сервер возвращает 401, сообщение отображается реактивно через `ref(error)`.

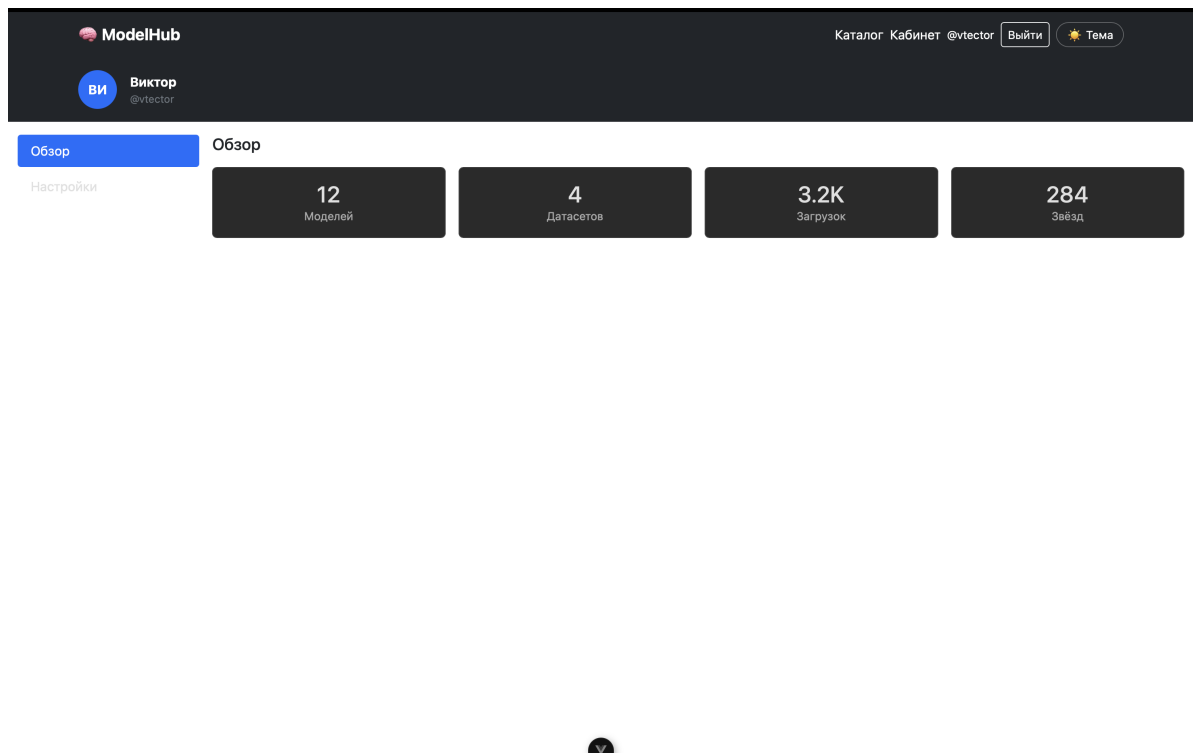


Рис. 5: Личный кабинет. DashboardView.vue. Защищённый маршрут — без токена navigation guard перенаправляет на `/login`.

Вывод

В ходе лабораторной работы приложение AI Model & Dataset Hub мигрировано на фреймворк Vue.js.

Что реализовано:

- 6 страниц-представлений (views) подключённых через Vue Router

- `Navigation guard` — защита маршрута `/dashboard` от неавторизованных пользователей
- Компонент `ModelCard.vue` — переиспользуется на 3 страницах
- Composable `useAuth.js` — общее состояние авторизации для всего приложения
- Composable `useApi.js` — `axios` с `interceptor` для автоматической подстановки токена
- Реактивность через `ref()` и `computed()` — DOM обновляется автоматически
- Тёмная/светлая тема через CSS-переменные с сохранением в `localStorage`

Преимущества Vue перед ванильным JS: компонентная архитектура упрощает поддержку кода, реактивность устраняет ручное обновление DOM, роутер обеспечивает настоящую SPA-навигацию.